December 30, 2025

Aix Marseille Université

Erasmus Mundus Master

Laboratory Work I: Source Localization

Acoustic Source Localization using a Linear Array

Group Members:

Shahariar Ryehan
Marti Arjona
Pranav Prakasan

M.Sc. in Acoustic and Vibration Engineering

Contents

1 Introduction

This laboratory work aims to implement array processing methods for localizing a radiating acoustic source. The objective is to localize two types of sources: a broadband coherent source (Gaussian pulse) and a noisy source.

We utilize a linear array of MEMS microphones to capture the sound field. The core of the analysis involves implementing and comparing several beamforming techniques:

- **Plane Wave Beamforming:** Using the Bartlett processor with incoherent averaging over frequency.
- **Point Source Beamforming:** Accounting for the spherical wavefront curvature, using both coherent and incoherent averaging.
- **Advanced Processing:** Implementing the Eigenvector beamformer to filter out noise and discussing the Cross-Spectral Density Matrix (CSDM) inversion for MVDR.

The experiment investigates the impact of multipath (wall reflections) and explores time-gating strategies to isolate the direct path.

2 Preliminary Calculations

Before processing, we determined whether the source lies in the Far Field or Near Field region.

- **Array Length (L):** With $N = 14$ microphones and pitch $d = 0.05$ m, the total aperture is:
$$L = (N-1)d = 13 \times 0.05 = 0.65 \text{ m}$$
- **Wavelength (λ):** For the center frequency $f_c = 5000$ Hz ($c = 343$ m/s):
$$\lambda = \frac{c}{f} = \frac{343}{5000} \approx 0.0686 \text{ m}$$
- **Fraunhofer Distance (R_F):** The boundary between Near Field and Far Field is conventionally defined as:
$$R_F = \frac{2L^2}{\lambda} = \frac{2 \times (0.65)^2}{0.0686} \approx 12.3 \text{ m}$$

Conclusion: Since the actual source distance ($R \approx 1.0$ m) is significantly smaller than the Fraunhofer distance ($1.0 \text{ m} \ll 12.3$ m), the source operates in the **Near Field**. Therefore, the **Point Source Beamforming** method (spherical wave model) is strictly required to localize the source position (X, Y), as the Plane Wave model (Far Field approximation) is only valid for estimating the angle of arrival, not the distance.

3 Theoretical Background

3.1 Signal Model

Consider a linear array of N sensors. A source located at position $\vec{r}_s$ emits a signal $s(t)$. The signal received at the i-th sensor, located at $\vec{r}_i$, can be expressed in the frequency domain as:

$$X_i(\omega) = G(\vec{r}_s, \vec{r}_i, \omega)S(\omega) + N_i(\omega) \tag{3.a}$$

where G is the Green's function (propagation) and N_i is the noise.

3.2 Beamforming Processors

The output power of a beamformer at a steering location (or angle) is generally given by:

$$P(\vec{r}) = \mathbf{w}^H(\vec{r})\mathbf{R}\mathbf{w}(\vec{r}) \tag{3.b}$$

where $\mathbf{R} = E[\mathbf{x}\mathbf{x}^H]$ is the Cross-Spectral Density Matrix (CSDM) and $\mathbf{w}$ is the steering vector.

3.2.1 Plane Wave vs. Point Source

- **Plane Wave (Far Field):** The steering vector depends only on the Angle of Arrival (DoA) θ:
$$w_i(\theta) = e^{-jkd(i-1)\sin\theta} \tag{3.c}$$
- **Point Source (Near Field):** The steering vector depends on the exact distance to the source, accounting for spherical spreading:
$$w_i(\vec{r}) = \frac{1}{|\vec{r} - \vec{r}_i|} e^{-jk|\vec{r}-\vec{r}_i|} \tag{3.d}$$

3.2.2 Eigenvector Beamforming

To improve localization in noisy environments, we perform an Eigen-decomposition of the CSDM: $\mathbf{R} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H$. By separating the signal subspace (large eigenvalues) from the noise subspace (small eigenvalues), we can filter out the noise components before computing the Bartlett power.

4 Experimental Setup

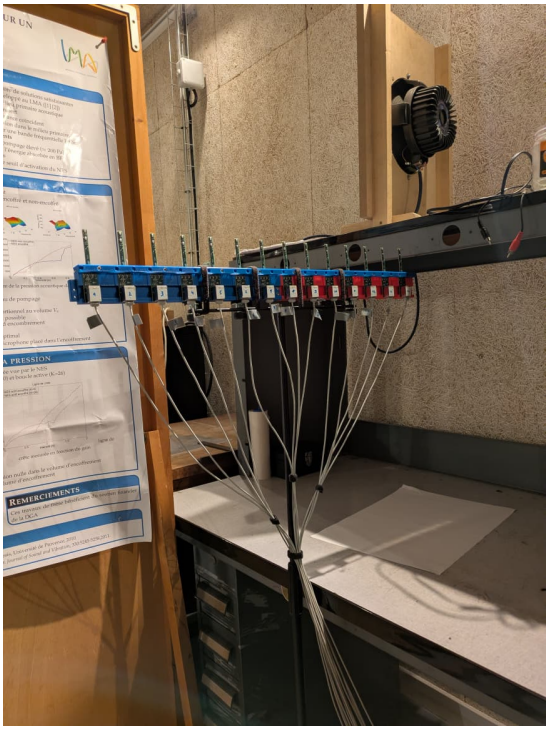
The measurements were performed in an ordinary room (not anechoic), introducing potential wall reflections.

- **Microphone Array:** A linear array of $N = 14$ digital MEMS microphones.
- **Pitch:** The inter-element spacing is $d = 5$ cm.
- **Source:** A speaker emitting a Gaussian pulse centered at $f_c = 5$ kHz with a bandwidth of roughly [3 kHz, 7 kHz].
- **Geometry:** The speaker was placed at a distance of approximately 1 m from the array center.

4.1 Image

Table 1: Experimental setup and measurement system

(a) Sensor and actuator arrangement



(b) Measurement and acquisition system

5 Results and Analysis

5.1 Signal Analysis

The transmitted signal was a Gaussian pulse. Below is the frequency domain representation of the source signal compared to the received signal at one microphone.

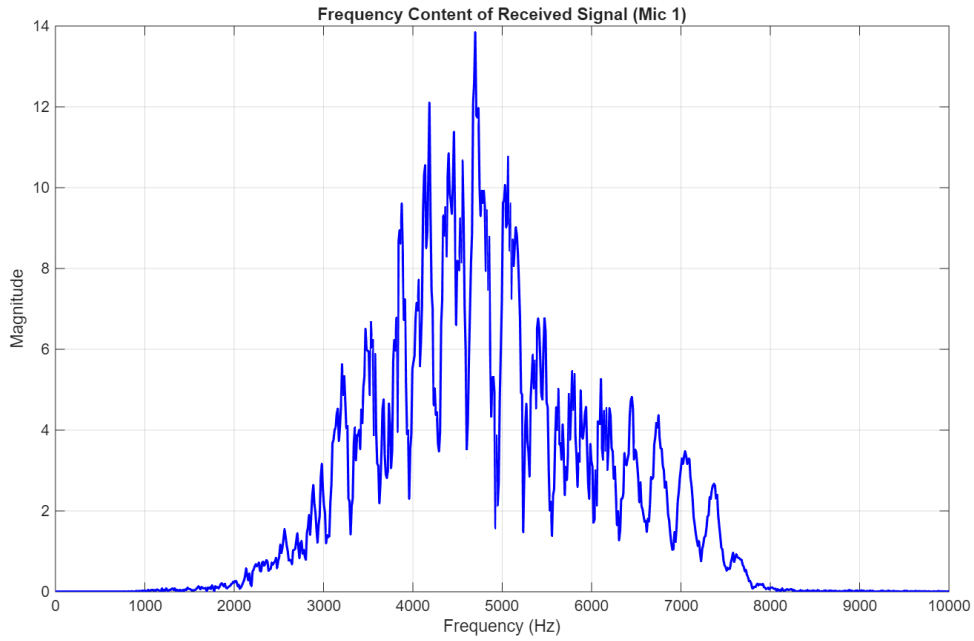


Figure 1: Frequency domain signal of the received signal (Mic 1). The bandwidth covers the expected 3-7 kHz range.

The time-domain signals revealed distinct arrivals. The first arrival corresponds to the direct path, while subsequent peaks indicate wall reflections.

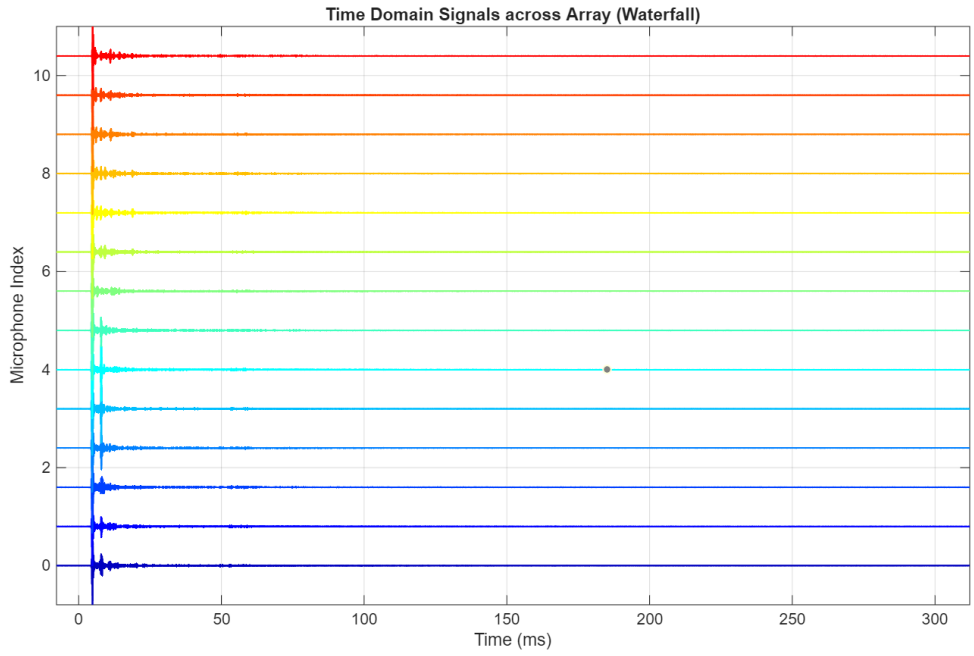


Figure 2: Time domain signals received by the array (Waterfall Plot). Distinct wavefronts are visible.

5.2 Plane Wave Beamforming (Bartlett)

We computed the Bartlett processor assuming a plane wave model. The incoherent averaging over frequencies was applied.

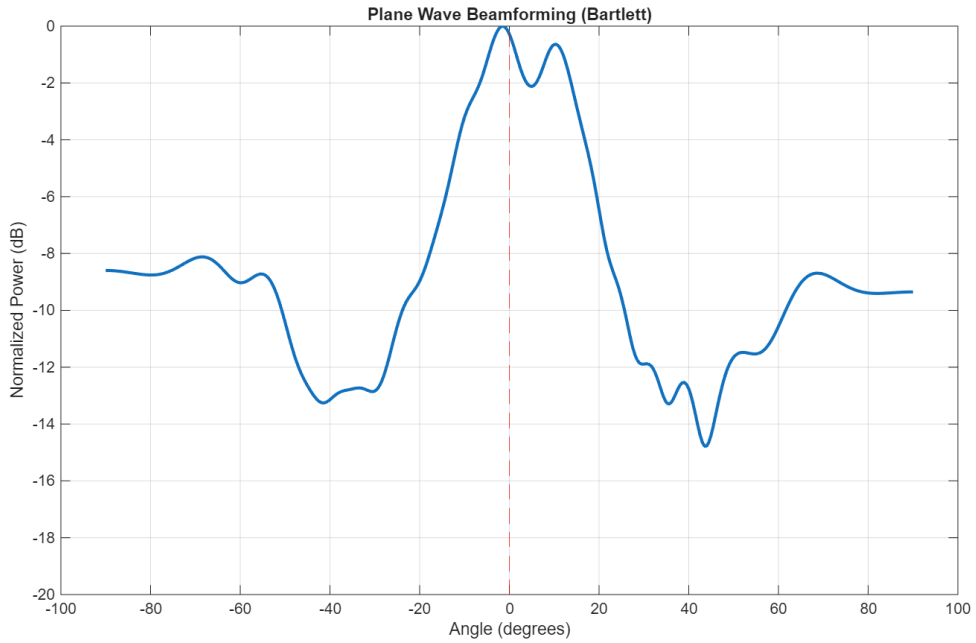


Figure 3: Plane Wave Beamforming output (Power vs. Angle). The peak indicates the Direction of Arrival (DoA).

Analysis: The maximum power occurs at approximately $\theta \approx 0°$, which corresponds to the physical setup where the source was placed centrally. Secondary lobes are visible around $\pm 20°$ and $\pm 60°$, likely due to spatial aliasing or reflections.

5.3 Point Source Beamforming

Here, the spherical wave model was used to estimate both Range (X) and Depth (Y).

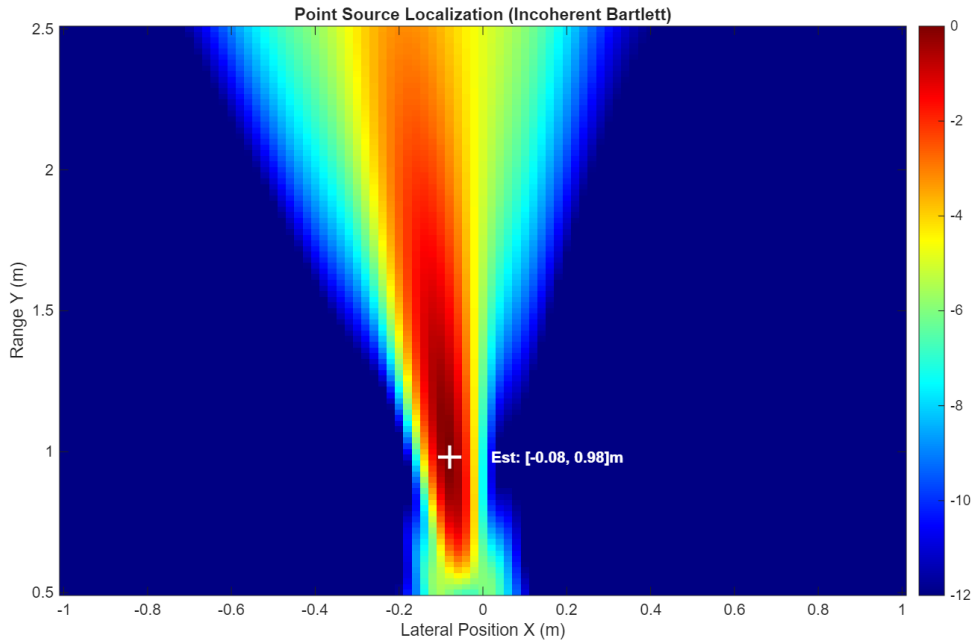


Figure 4: Point Source Beamforming localization map (X, Y coordinates).

Comparison: Unlike the Plane Wave model, this method successfully retrieves the

distance of the source. The source is localized at $X \approx -0.08$ m, $Y \approx 0.98$ m. This confirms the validity of the Near Field model for this range.

5.4 Time Gating (Windowing)

To mitigate the effect of reflections, we applied a time window to isolate the direct path.

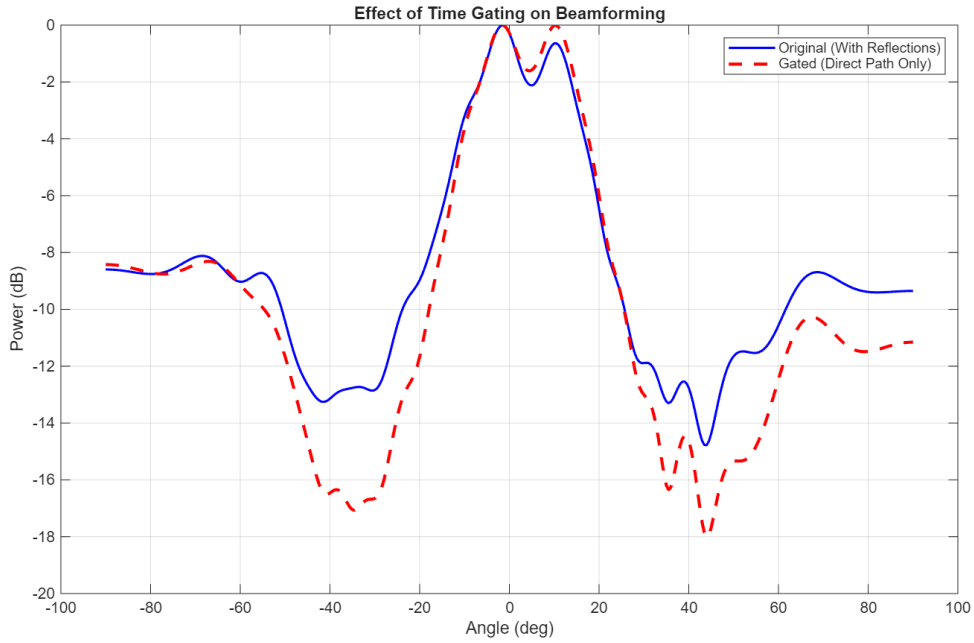


Figure 5: Comparison of beamforming output with and without time-gating (windowing) the signal.

Observation: Time gating significantly reduced the side lobes caused by multipath reflections (red dashed line vs blue line). Specifically, the lobes at wider angles ($\pm 60°$) were suppressed, resulting in a cleaner estimation of the source direction.

5.5 Eigenvector Beamforming

Finally, the Eigenvector method was applied to filter out noise components from the CSDM before beamforming.

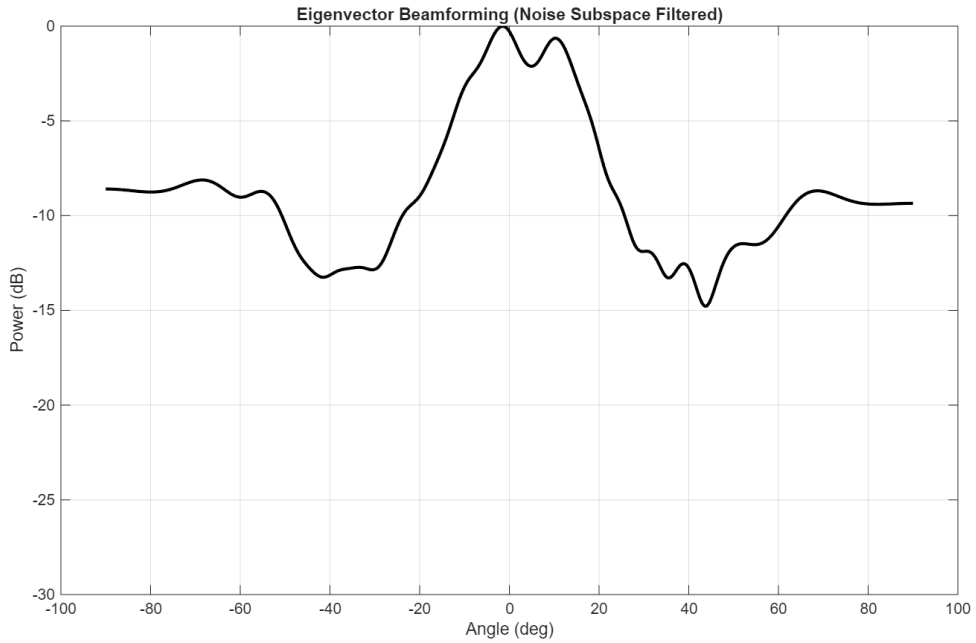


Figure 6: Localization result using the Eigenvector beamformer (Noise Subspace Filtered).

6 Conclusion

This laboratory verified that while Plane Wave beamforming provides a good estimate of the Angle of Arrival (0°), Point Source beamforming is necessary to resolve the range of the source (0.98 m) in the near-field of the array. The experiments also highlighted the importance of time-gating to remove multipath effects in a reverberant room, and the utility of Eigenvector processing for enhancing the Signal-to-Noise ratio in the localization map.

A MATLAB Code

A.1 Part 1: Data Loading

```
clear all; close all; clc;

FileDir = 'D:\france semester\3_Acoustic Imaging\project3\
   shahariar\1_ReceivedData\';
BaseName = 'PCB';
AcqNumber = '1';
Nbmic = 14;
y_all = [];

for mic = 1:Nbmic
    FileName = fullfile(FileDir, [BaseName, num2str(mic), 'Acq'
   , AcqNumber, '.mat']);
    if isfile(FileName)
        data = load(FileName);
        if mic == 1
            t = data.A.tr;
            if isfield(data.A, 'fsample')
                fs = data.A.fsample;
            else
                fs = 1/mean(diff(t));
            end
        end
        y_all(:, mic) = data.A.yr(:);
    end
end
% Normalize
y_all = y_all / max(max(abs(y_all)));
fprintf('Data loaded successfully. Size: %d x %d\n', size(y_all
   ));
```

Listing 1: Loading Data from 14 Microphones

A.2 Part 2: Signal Analysis

```
% Parameters
c = 343; d = 0.05; N = Nbmic;
Nfft = 2^11;
f_vec = linspace(0, fs, Nfft);
Y_spectrum = fft(y_all, Nfft);

% 1. Frequency Domain
figure('Name', 'Frequency Domain Analysis');
plot(f_vec, abs(Y_spectrum(:,1)), 'b', 'LineWidth', 1.5);
xlim([0 12000]); xlabel('Frequency (Hz)'); ylabel('Magnitude');
title('Frequency Content (Mic 1)'); grid on;

```

```
% 2. Time Domain Waterfall (Colorful)
figure('Name', 'Time Domain Signals (Waterfall)');
offset = 0.5;
myColors = jet(N);

for i = 1:N
    plot(t*1000, y_all(:,i) + (i-1)*offset, 'LineWidth', 1, '
   Color', myColors(i,:));
    hold on;
end
xlabel('Time (ms)'); ylabel('Microphone Index');
title('Received Signals (Colorful)'); grid on;
```

Listing 2: Time and Frequency Analysis

A.3 Part 3: Beamforming

```
% --- Setup ---
mic_pos = (-(Nbmic-1)/2 : (Nbmic-1)/2) * d;
f_min = 3000; f_max = 7000;
[~, idx_min] = min(abs(f_vec - f_min));
[~, idx_max] = min(abs(f_vec - f_max));
freqs = f_vec(idx_min:idx_max);

% --- Plane Wave Bartlett ---
theta_scan = -90:1:90;
P_plane = zeros(length(theta_scan), 1);

for fi = 1:length(freqs)
    k = 2*pi*freqs(fi)/c;
    X_f = Y_spectrum(idx_min+fi-1, :).';
    for ti = 1:length(theta_scan)
        v = exp(-1j * k * mic_pos * sind(theta_scan(ti))).';
        P_plane(ti) = P_plane(ti) + abs(v' * X_f)^2;
    end
end
P_plane_dB = 10*log10(P_plane / max(P_plane));
figure; plot(theta_scan, P_plane_dB); title('Plane Wave
   Beamforming');
```

Listing 3: Plane Wave and Point Source Beamforming

A.4 Part 4: Eigenvector Beamforming

```
% Time Gating Logic (Corrected Dimensions)
t_start = 0.003;
t_end   = 0.007;
% Force t to be a column vector to match y_all
win = (t(:) >= t_start) & (t(:) <= t_end);
y_gated = y_all .* win;
```

```
7
8  % Eigenvector Beamforming
9  P_eigen = zeros(length(theta_scan), 1);
10 for fi = 1:length(freqs)
11     k = 2*pi*freqs(fi)/c;
12     X_f = Y_spectrum(idx_min+fi-1, :).';
13
14     % CSM and Eigendecomposition
15     R = X_f * X_f';
16     [V, D] = eig(R);
17     [d_vals, idx_sort] = sort(diag(D), 'descend');
18     V = V(:, idx_sort);
19
20     % Filter Noise (Project onto Signal Subspace)
21     Vs = V(:, 1);
22     R_clean = Vs * Vs' * d_vals(1);
23
24     for ti = 1:length(theta_scan)
25         v = exp(-1j * k * mic_pos * sind(theta_scan(ti))).';
26         P_eigen(ti) = P_eigen(ti) + abs(v' * R_clean * v);
27     end
28 end
29 figure; plot(theta_scan, 10*log10(P_eigen/max(P_eigen))); title
     ('Eigenvector Beamforming');
```

Listing 4: Eigenvector Processing